

一、基本语法以及一些特殊函数

1.ASCII表

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

```
ord('A')==65,chr(65+32)=='a','A'.lower()->'a','a'.upper()->'A'
```

2.counter()

记录各个元素记录的个数，本质是字典

```
from collections import Counter
a = [12, 3, 4, 3, 5, 11, 12, 6, 7]
x=Counter(a)
for i in x.keys():
    print(i, ":", x[i])
x_keys = list(x.keys()) #[12, 3, 4, 5, 11, 6, 7]
x_values = list(x.values()) #[2, 2, 1, 1, 1, 1, 1]
for i in x.elements():
    print ( i, end = " ") #[12,12,3,3,4,5,11,6,7]
c=Counter('121312334352123125555555')
cc=sorted(c.items(),key=lambda x:x[1],reverse=True)
#[('5', 9), ('1', 5), ('2', 5), ('3', 5), ('4', 1)]
```

3.enumerate() 索引和函数一起出现

```
seasons = ['Spring', 'Summer', 'Fall', 'Winter']
print(list(enumerate(seasons)))
##[(0, 'Spring'), (1, 'Summer'), (2, 'Fall'), (3, 'Winter')]
for i,element in enumerate(seasons):
    print(i,element)
```

4.判断质数：（只用最快的线性筛）

```
def euler_sieve(n):
    is_prime = [True] * (n + 1)
    primes = []
    for i in range(2, n + 1):
        if is_prime[i]:
            primes.append(i)
        for p in primes:
            if i * p > n:
                break
            is_prime[i * p] = False
            if i % p == 0:
                break
    return primes
```

5.cmp_to_key

```
from functools import cmp_to_key
def compar(a,b):
    if a>b:
        return 1#大的在后
    if a<b:
        return -1#小的在前
    else:
        return 0#返回零不变位置
l=[1,5,2,4,6,7,6]
l.sort(key=cmp_to_key(compar))
print(l)#[1,2,4,5,6,6,7]
```

6.sys读入输出

取代input(),print()

```
import sys
#输入
input = sys.stdin.read
data = input().split()
#输出
sys.stdout.write('\n'.join(map(str, results)) + '\n')
```

还可以替代：

```
while(True):
    try:
        ...
    except EOFError:
        break
```

7.defaultdict

有默认值的字典

```
from collections import defaultdict
d = defaultdict(int)
L = [1, 2, 3, 4, 2, 4, 1, 2]
for i in L:
    d[i] += 1
print(d)##defaultdict(<class 'int'>, {1: 2, 2: 3, 3: 1, 4: 2})

d = defaultdict(list)
for i in range(5):
    d[i].append(i)
print(d)##defaultdict(<class 'list'>, {0: [0], 1: [1], 2: [2], 3: [3], 4: [4]})

d = defaultdict(lambda: "Not Present")
d["a"] = 1
d["b"] = 2
print(d["a"])#1
print(d["b"])#2
print(d["c"])#Not Present
```

8.保留小数

```
number = 3.14159
formatted_number = "{:.2f}".format(number)
print(formatted_number) # 输出: 3.14
```

9.全排列:

```
from itertools import permutations
# Get all permutations of [1, 2, 3]
perm = permutations([1, 2, 3])
# Print the obtained permutations
for i in list(perm):
    print (i)
```

```
from itertools import permutations
# Get all permutations of length 2
# and length 2
perm = permutations([1, 2, 3], 2)
# Print the obtained permutations
for i in list(perm):
    print (i)
```

10.进制转换:

```
hex(1033)
'0x409'
bin(10)
'0b1010'
oct(11)
'013'
int('10100111110',2)
```

二、三种常用数据结构

1.只有一端能进出stack

使用list，并且只用pop(),append()进行出栈入栈即可实现

2.两端都能进出的deque

两端进出都是O(1)

```
from collections import deque
de=deque([1,2,3])
de.append(4)
de.appendleft(5)
de.extend([6,7,8])
de.extendleft([9,10,11])
de.pop()
de.popleft()
print(de)##deque([10, 9, 5, 1, 2, 3, 4, 6, 7])
```

3.维护第一个最小的小顶堆heapq

```
import heapq
li = [5, 7, 9, 1, 3]
heapq.heapify(li)
heapq.heappush(li,4)
heapq.heappop(li) #1
print(li) #[3, 5, 4, 7, 9]
n_smallest = heapq.nsmallest(3, li) #[3,4,5]
n_largest = heapq.nlargest(2, li) #[9,7]
```

三、贪心和矩阵

贪心算法是用来解决一类最优化问题，并希望由局部最优策略来推得全局最优结果

双指针（略）和二分查找是贪心算法中常用的技巧

对于矩阵有加保护圈这一技巧（太简单略）

1.二分查找:

```
def bisect_right(a,x):
    hi=len(a)
    lo=0
    while lo<hi:
        mid=(lo+hi)//2
        if x<a[mid]:
            hi=mid
        else:
            lo=mid+1
    return lo
def bisect_left(a,x):
    hi=len(a)
    lo=0
    while lo<hi:
        mid=(lo+hi)//2
        if x<=a[mid]:
            hi=mid
        else:
            lo=mid+1
    return lo
a=[1,2,2,2,3,4]
x=2
print(bisect_left(a,x),bisect_right(a,x))#1,4
```

也可以直接引库

```
import bisect
bisect.bisect(a,x)
bisect.bisect_left(a,x)
bisect.insort(a,x)(直接利用insect(lo,x))
bisect.bisect_left(a,x)
```

2.经典案例:

1.区间合并

给出一堆区间，要求合并所有有交集的区间（端点处相交也算有交集）。最后问合并之后的区间。

【步骤一】：按照区间左端点从小到大排序。

【步骤二】：维护前面区间中最右边的端点为 ed 。从前往后枚举每一个区间，判断是否应该将当前区间视为新区间。

假设当前遍历到的区间为第 i 个区间 $[l_i, r_i]$ ，有以下两种情况：

- $l_i \leq ed$: 说明当前区间与前面区间有交集。因此不需要增加区间个数，但需要设置 $ed = \max(ed, r_i)$ 。
- $l_i > ed$: 说明当前区间与前面没有交集。因此需要增加区间个数，并设置 $ed = \max(ed, r_i)$ 。

```
intervals.sort()
res = []
st, ed = -sys.maxsize, -sys.maxsize
for v in intervals:
    if ed == -sys.maxsize:
        st, ed = v[0], v[1]
```

```

elif v[0] <= ed:
    ed = max(v[1], ed)
elif v[0] > ed:
    res.append([st, ed])
    st, ed = v[0], v[1]
if ed != -sys.maxsize:
    res.append([st, ed])
return res

```

2.选择不相交区间

给出一堆区间，要求选择尽量多的区间，使得这些区间互不相交，求可选取的区间的最大数量。这里端点相同也算有重复。

【步骤一】：按照区间右端点从小到大排序。

【步骤二】：从前往后依次枚举每个区间。

假设当前遍历到的区间为第 i 个区间 $[l_i, r_i]$ ，有以下两种情况：

- $l_i \leq ed$: 说明当前区间与前面区间有交集。因此直接跳过。
- $l_i > ed$: 说明当前区间与前面没有交集。因此选中当前区间，并设置 $ed = r_i$ 。

```

intervals.sort(key=lambda x: x[1])
res = 0
ed = -sys.maxsize
for v in intervals:
    if ed < v[0]:
        res += 1
        ed = v[1]
print(res)

```

3.区间选点问题

给出一堆区间，取尽量少的点，使得每个区间内至少有一个点（不同区间内含的点可以是同一个，位于区间端点上的点也算作区间内）。

这个题可以转化为上一题的求最大不相交区间的数量。

对于这些最大的不相交区间，肯定是每个区间都需要选出一个点。而其他的区间都是和这些选出的区间有重复的，我们只需要把点的位置选在重合的部分即可。

也可以换一种思路：

我们将区间按照右端点从小到大排序，这时我们应该尽量选择当前区间最右边的点。

因为最右边的点可能和下面的其他区间重复，所以至少不比选择区间靠前位置的点差。

所以，最后的解法与选择不相交区间问题解法完全一样。

4.区间覆盖问题

给出一堆区间和一个目标区间，问最少选择多少区间可以覆盖掉题中给出的这段目标区间。

【步骤一】：按照区间左端点从小到大排序。

步骤二】：从前往后依次枚举每个区间，在所有能覆盖当前目标区间起始位置start的区间之中，选择右端点最大的区间。

假设右端点最大的区间是第 i 个区间，右端点为 r_i 。

最后将目标区间的start更新成 r_i

```
cnt=0
st=0
ed=time
clips.sort()
i=0
while True:
    edm=st
    while i<len(clips) and clips[i][0]<=st:
        edm=max(edm,clips[i][1])
        i=i+1
    if edm==st:
        return -1
    st=edm
    cnt+=1
    if st>=ed:
        return cnt
```

5.区间分组问题

给出一堆区间，问最少可以将这些区间分成多少组使得每个组内的区间互不相交。

【步骤一】：按照区间左端点从小到大排序。

【步骤二】：从前往后依次枚举每个区间，判断当前区间能否被放到某个现有组里面。

(即判断是否存在某个组的右端点在当前区间之中。如果可以，则不能放到这一组)

假设现在已经分了 m 组了，第 k 组最右边的一个点是 r_k ，当前区间的范围是 $[L_i, R_i]$ 。则：

如果 $L_i \leq r_k$ 则表示第 i 个区间无法放到第 k 组里面。反之，如果 $L_i > r_k$ ，则表示可以放到第 k 组。

- 如果所有 m 个组里面没有组可以接收当前区间，则当前区间新开一个组，并把自己放进去。
- 如果存在可以接收当前区间的组 k ，则将当前区间放进去，并更新当前组的 $r_k = R_i$ 。

注意：

为了能快速的找到能够接收当前区间的组，我们可以使用**优先队列（小顶堆）**。

优先队列里面记录每个组的右端点值，每次可以在 $O(1)$ 的时间拿到右端点中的最小值。

```

import heapq
l=[[1, 5], [2, 7], [4, 5]]
q=[]
for st,ed in l:
    if not q or st<q[0]:
        heapq.heappush(q,ed)
    else:
        heapq.heappop(q)
        heapq.heappush(q,ed)
print(len(q))

```

6.最长下降子序列

```

from bisect import bisect_right
def min_testers_needed(scores):
    scores.reverse() # 反转序列以找到最长下降子序列的长度
    lis = [] # 用于存储最长上升子序列
    for score in scores:
        pos = bisect_right(lis, score)
        if pos < len(lis):
            lis[pos] = score
        else:
            lis.append(score)
    return len(lis)
N = int(input())
scores = list(map(int, input().split()))
result = min_testers_needed(scores)
print(result)

```

附dp解法: $O(n^2)$

```

n = int(input())
*b, = map(int, input().split())
dp = [1]*n
for i in range(1, n):
    for j in range(i):
        if b[j] < b[i]:
            dp[i] = max(dp[i], dp[j]+1)
print(max(dp))

```

四、递归:

递归是一种解决问题的方法，它涉及将一个问题分解成越来越小的子问题，直到得到一个足够小的问题，可

以轻易地解决。递归涉及到一个函数调用自身。虽然表面上看起来可能没什么特别之处，但递归使我们能够

编写出优雅的方案，来解决那些可能非常难以编程的问题。

1.递归三法则

1.递归算法必须有一个基准情形

2.递归算法必须改变其状态并朝着基准情形前进。

3.递归算法必须调用自身，即进行递归调用。

2.典中典

1.八皇后

```
ans = []
def queen_dfs(A, cur=0): #考虑放第cur行的皇后
    if cur == len(A): #如果已经放了n个皇后，一组新的解产生了
        ans.append(''.join([str(x+1) for x in A])) #注意避免浅拷贝
        return
    for col in range(len(A)): #将当前皇后逐一放置在不同的列，每列对应一组解
        for row in range(cur): #逐一判定，与前面的皇后是否冲突
            #因为预先确定所有皇后一定不在同一行，所以只需要检查是否同列，或者在同一斜线上
            if A[row] == col or abs(col - A[row]) == cur - row:
                break
        else: #若都不冲突
            A[cur] = col #放置新皇后，在cur行，col列
            queen_dfs(A, cur+1) #对下一个皇后位置进行递归
queen_dfs([None]*8)
for _ in range(int(input())):
    print(ans[int(input()) - 1])
```

2.全排列

```
hashTable = [False] * maxn # 当整数i已经在数组 P中时为 true
#@recviz
def increasing_permutations(n, prefix=[]):
    if len(prefix) == n: # 递归边界，已经处理完排列的1~位
        return [prefix]
    result = []
    for i in range(1, n + 1):
        if hashTable[i]:
            continue
        hashTable[i] = True # 记i已在prefix中
        # 把i加入当前排列，处理排列的后续号位
        result += increasing_permutations(n, prefix + [i])
        hashTable[i] = False # 处理完为i的子问题，还原状态
    return result
n = int(input())
result = increasing_permutations(n)
for r in result:
    print(' '.join(map(str,r)))
```

3.递归程序优化两板斧

1.增加递归深度限制

```
import sys
sys.setrecursionlimit(1 << 30) # 将递归深度限制设置为 2^30
```

2.缓存中间结果

之间建个列表or字典调用或者直接内置函数

缓存中间结果缓存中间结果缓存中间结果

```
from functools import lru_cache
@lru_cache(maxsize=None)
```

五、动态规划(dp)

动态规划(Dynamic Programming, DP)是一种用来解决一类最优化问题的算法思想。简单来说, 动态规划将一个复杂的问题分解成若干个子问题, 通过综合子问题的最优解来得到原问题的最优解。需要注意的是, 动态规划会将每个求解过的子问题的解记录下来, 这样当下一次碰到同样的子问题时, 就可以直接使用之前记录的结果, 而不是重复计算。一般可以使用递归或者递推的写法来实现动态规划, 其中递归写法在此处又称作记忆化搜索。

如果一个问题可以被分解为若干个子问题, 且这些子问题会重复出现, 那么就称这个问题拥有重叠子问题。动态规划通过记录重叠子问题的解, 来使下次碰到相同的子问题时直接使用之前记录的结果以此避免大量重复计算。因此, 一个问题必须拥有重叠子问题, 才能使用动态规划去解决。

如果一个问题最优解可以由其子问题的最优解有效地构造出来, 那么就称这个问题拥有最优子结构。需要指出, 一个问题必须拥有重叠子问题和最优子结构, 才能使用动态规划去解决。

dp是由状态转移方程和底端结果而得到的, 事实上, 如何设计状态和状态转移方程, 才是动态规划的核心, 而它们也是动态规划最难的地方。

1.最大连续子序列和

dp方法

```
n = int(input())
*a, = map(int, input().split())
dp = [0]*n
dp[0] = a[0]
for i in range(1, n):
    dp[i] = max(dp[i-1]+a[i], a[i])
print(max(dp))
```

Kadane's 算法

```
def max_subarray_sum(arr):
    if not arr:
        return 0
    max_current = max_global = arr[0]
    for num in arr[1:]:
        max_current = max(num, max_current + num)
        if max_current > max_global:
            max_global = max_current
    return max_global
# 测试用例
arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print("最大子数组和为:", max_subarray_sum(arr)) # 输出: 最大子数组和为: 6
```

变体（土豪购物）双dp

```
a=list(map(int,input().split(",")))
n=len(a)
dp1,dp2=a[0],a[0]
ans=0
for i in range(1,n):
    dp1,dp2=max(a[i],dp1+a[i]),max(dp1,dp2+a[i])
    ans=max(ans,dp2)
print(ans)
```

2.背包

1.0-1背包

n个物品，b容量，价值p，重量w

```
n,b=map(int,input().split())
p=list(map(int,input().split()))
w=list(map(int,input().split()))
dp=[0]*(b+1)
for i in range(n):
    for j in range(b,w[i]-1,-1):
        dp[j]=max(dp[j],dp[j-w[i]]+p[i])
print(dp[-1])
```

2.完全背包

只需修改遍历顺序

```
n,b=map(int,input().split())
p=list(map(int,input().split()))
w=list(map(int,input().split()))
dp=[0]*(b+1)
for i in range(n):
    ***for j in range(w[i], b+1):***##(从小容量到大容量遍历，即可装多个物品)
        dp[j]=max(dp[j],dp[j-w[i]]+p[i])
print(dp[-1])
```

3.必须填满完全背包(剪丝带)

只需修改初始dp数值 (只有剩0容量为0, 剩下都是-float('inf'))

```
n,a,b,c=map(int,input().split())
***dp=[0]+[-float('inf')]*n***##(必须一点不剩才有真值)
for i in (a,b,c):
    for j in range(i,n+1):
        dp[j]=max(dp[j],dp[j-i]+1)
print(dp[-1])
```

4.多重背包(每个物品有上限)

最简单的思路是将多个同样的物品看成多个不同的物品, 从而化为0-1背包。稍作优化: 可以改善拆分方式, 譬如将m个1拆成 $x_1, x_2, \dots, x_t$ 个1, 只需要这些 x_i 中取若干个的和能组合出1至m即可。最高效的拆分方式是尽可能拆成2的幂, 也就是所谓“二进制优化”

六、搜索(dfs, bfs)

1、dfs

从起点开始前进, 当碰到岔道口时, 总是选择其中一条岔路前进(例如图中总是先选择最右手边的岔路), 在岔路上如果又遇到新的岔道口, 仍然选择新岔道口的其中一条岔路前进, 直到碰到死胡同才回退到最近的岔道口选择另一条岔路。也就是说, 当碰到岔道口时, 总是以“深度”作为前进的关键词, 不碰到死胡同就不回头, 因此把这种搜索的方式称为深度优先搜索(Depth First Search, DFS)。

从迷宫的例子还应该注意, 深度优先搜索会走遍所有路径, 并且每次走到死胡同就代表一条完整路径的形成。这就是说, 深度优先搜索是一种枚举所有完整路径以遍历所有情况的搜索方法。

1、数个数 (数湖)

主要思路是设置一个函数把一个湖全变成陆, 然后数全变成陆使用了几次这个函数即可

```
import sys
sys.setrecursionlimit(1<<30)
n,m=map(int,input().split())
l=[list(input()) for _ in range(n)]
d=[(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]
cnt=0
def dfs(x,y):
    l[x][y]='.'
    for dx,dy in d:
        nx=x+dx
        ny=y+dy
        if 0<=nx<n and 0<=ny<m:
            if l[nx][ny]=='w':
                dfs(nx,ny)
for i in range(n):
    for j in range(m):
        if l[i][j]=='w':
            dfs(i,j)
            cnt+=1
print(cnt)
```

2.迷宫问题:

用visit矩阵解决重复路径问题

```
n,m=map(int,input().split())
l=[list(map(int,input().split())) for i in range(n)]
cnt=0
d=[(0,1),(0,-1),(1,0),(-1,0)]
visit=[[False for i in range(m)] for j in range(n)]
def dfs(x,y):
    if (x,y)==(n-1,m-1):
        global cnt
        cnt+=1
    else:
        for i,j in d:
            nx=x+i
            ny=y+j
            if 0<=nx<n and 0<=ny<m and l[nx][ny]==0 and visit[nx][ny]==False:
                visit[nx][ny]=True
                dfs(nx,ny)
                visit[nx][ny]=False
visit[0][0]=True
dfs(0,0)
print(cnt)
```

2.bfs

接下来将介绍的广度优先搜索 (Breadth FirstSearch,BFS)则是以广度为第一关键词,当碰到岔道口时,总是先依次访问从该岔道口能直接到达的所有结点,然后再按这些结点被访问的顺序去依次访问它们能直接到达的所有结点,以此类推,直到所有结点都被访问为止。这就跟平静的水面中投入一颗小石子一样,水花总是以石子落水处为中心,并以同心圆的方式向外扩散至整个水面,从这点来看和 DFS 那种沿着一条线前进的思路是完全不同的。

广度优先搜索 (BFS)一般由队列实现,且总是按层次的顺序进行遍历

bfs一般在要找到某个最短的东西时使用

1.数个数

使用deque使得弹入弹出都是O(1)

```
from collections import deque
n,m=map(int,input().split())
l=[list(map(int,input().split())) for i in range(n)]
d=[(0,1),(0,-1),(1,0),(-1,0)]
cnt=0
inq=set()
def bfs(x,y):
    q=deque([])
    q.append([x,y])
    inq.add((x,y))
    while q:
        x,y=q.popleft()
        for i,j in d:
            nx=x+i
            ny=y+j
```

```

        if 0<=nx<n and 0<=ny<m and l[nx][ny]==1 and (nx,ny) not in inq:
            q.append([nx,ny])
            inq.add((nx,ny))

for i in range(n):
    for j in range(m):
        if l[i][j]==1 and (i,j) not in inq:
            bfs(i,j)
            cnt+=1
print(cnt)

```

2.迷宫最短路径

使用heapq以得到最短路径

```

import heapq
d=[(0,-1),(0,1),(1,0),(-1,0)]
n,m=map(int,input().split())
l=[list(map(int,input().split())) for i in range(n)]
q=[]
heapq.heappush(q,(0,0,0))
inq=set()
inq.add((0,0))
while q:
    s,x,y=heapq.heappop(q)
    if (x,y)==(n-1,m-1):
        print(s)
        break
    for i,j in d:
        nx,ny=x+i,y+j
        if 0<=nx<n and 0<=ny<m and l[nx][ny]!=1 and (nx,ny) not in inq:
            inq.add((nx,ny))
            heapq.heappush(q,(s+1,nx,ny))
    else:
        print(-1)

```

3.最短路径Dijkstra

以走山路为例

此算法即使用一个二维数组储存到达某处的最小用力，将判断是否到达变为判断是否用了更小的力，以找到到达终点所需最小力。

```

import heapq
n,m,p=map(int,input().split())
l=[list(input().split()) for i in range(n)]
d=[(0,1),(0,-1),(1,0),(-1,0)]
for _ in range(p):
    x1,y1,x2,y2=map(int,input().split())
    if l[x1][y1]=='#' or l[x2][y2]=='#':
        print("NO")
        continue
    q=[]
    force=[[float('inf') for i in range(m)] for j in range(n)]
    heapq.heappush(q,(0,x1,y1))

```

```

force[x1][y1]=0
while q:
    f,x,y=heapq.heappop(q)
    if (x,y)==(x2,y2):
        print(f)
        break
    for i,j in d:
        nx,ny=x+i,y+j
        if 0<=nx<n and 0<=ny<m and l[nx][ny]!='#':

            if f+abs(int(l[x][y])-int(l[nx][ny]))<force[nx][ny]:
                force[nx][ny]=f+abs(int(l[x][y])-int(l[nx][ny]))
                heapq.heappush(q,(force[nx][ny],nx,ny))

    else:
        print('NO')

```

七、特殊技巧

1.滑动窗口

滑动窗口是一个队列，比如例题中的 abcacbb，进入这个队列（窗口）为 abc 满足题目要求，当再进入 a，队列变成了 abca，这时候不满足要求。所以，我们要移动这个队列！如何移动？我们只要把队列的左边的元素移出就行了，直到满足题目要求！一直维持这样的队列，找出队列出现最长的长度时候！
时间复杂度：O(n)

```

d={}
ma=0
start=0
for i,ele in enumerate(s):
    if ele in d and d[ele]>=start:
        start=d[ele]+1
    d[ele]=i
    ma=max(ma,i-start+1)
return ma

```

2.并查集

用于存储非重叠或不相交子集元素的数据结构被称为不相交集数据结构（disjoint set data structure）。不相交集数据结构支持以下操作：

- 1.向不相交集中添加新的集合。
- 2.使用并（Union）操作将不相交集合并为一个单一的不相交集合。
- 3.使用查找（Find）操作找到不相交集的代表元素。
- 4.检查两个集合是否不相交。

所使用的数据结构：

数组：一个整数数组被称为 Parent[]。如果我们处理的是 N 个元素，那么数组的第 i 个元素代表第 i 个项。更准确地说，Parent[] 数组的第 i 个元素是第 i 个项的父节点。这些关系创建了一个或多个虚拟树。

树：它是一个不相交集（Disjoint set）。如果两个元素位于同一棵树中，则它们属于同一个不相交集。每棵树的根节点（或最顶层节点）称为该集合的代表元（representative）。每个集合总是有一个唯一的代表元。一个简单的规则来识别代表元是：如果 i 是一个集合的代表元，则 $\text{Parent}[i] = i$ 。如果 i 不是其集合的代表元，则可以通过沿着树向上遍历直到找到代表元来确定它。

我知道有点抽象，上例题：

现有一个学校，学校中有若干个班级，每个班级中有若干学生，每个学生只会存在于一个班级中。如果学生 A 和学生 B 处于一个班级，学生 B 和学生 C 处于一个班级，那么我们称学生 A 和学生 C 也处于一个班级。

现已知学校中共 n 个学生（编号为从 1 到 n ），并给出 m 组学生关系（指定两个学生处于一个班级），问总共有多少个班级。

输入

第一行两个整数 m 、 n ($1 \leq n \leq 100, 1 \leq m \leq 100$)，分别表示学生个数、学生关系个数；

接下来 m 行，每行两个整数 a 和 b ($1 \leq a \leq n, 1 \leq b \leq n, a \neq b$)，表示编号为 a 的学生和编号为 b 的学生处于一个班级。

输出

输出一个整数，表示班级个数。

样例1

输入

```
1 5 3
2 4 2
3 1 3
4 2 5
```

输出

```
1 2
```

```
n,m=map(int,input().split())
p=[i for i in range(n+1)]
def find(x):
    if p[x]==x:
        return x
    else:
        p[x]=find(p[x])
        return p[x]
def union(x,y):
    p[find(x)]=find(y)
for _ in range(m):
    x,y=map(int,input().split())
    union(x,y)
se={find(x) for x in range(1,n+1)}
print(len(se))
```

若要给出班级人数则可给出人数数组size，在合并时也将人数合并

```
n,m=map(int,input().split())
p=[i for i in range(n+1)]
size=[1 for i in range(n+1)]
def find(x):
```



```

    if p[x]==x:
        return x
    else:
        p[x]=find(p[x])
        return p[x]
def union(x,y):
    x1=find(x)
    y1=find(y)
    if x1!=y1:
        p[x1]=y1
        size[y1]+=size[x1]
for _ in range(m):
    x,y=map(int,input().split())
    union(x,y)
se=[size[x] for x in range(1,n+1) if x==p[x]]
se.sort(reverse=True)
print(len(se))
print(' '.join(map(str,se)))

```

3.单调栈

即人为控制栈内元素单调，找某侧最近一个比其大的值，使用单调栈维持栈内元素递减；找某侧最近一个比其小的值使用单调栈，维持栈内元素递增

例子：接雨水

在这道题，由于需要找某个位置两侧比其高的柱子（只有两侧有比当前位置高的柱子，当前位置才能接雨水），我们可以维持栈内元素的单调递减。

```

stack=[]
water=0
n=len(height)
for i in range(n):
    while stack and height[stack[-1]]<height[i]:
        top=stack.pop()
        if not stack:
            break
        d=i-stack[-1]-1
        h=min(height[stack[-1]],height[i])-height[top]
        water+=d*h
    stack.append(i)
return water

```

滑动窗口最大值（长度为k的滑动窗口，给出每次窗口中最大值）：同样是维护递减双端序列

```

from collections import deque
res=[]
de=deque()
if not nums or k==0:
    return []
if k==1:
    return nums

```

```
for i in range(len(nums)):
    if de and de[0]<i-k+1:
        de.popleft()
    while de and nums[de[-1]]<nums[i]:
        de.pop()
    de.append(i)
    if i>=k-1:
        res.append(nums[de[0]])
return res
```